DT LAB-1

Section: C1
Name:G.Susritha
Roll num: 160123733014

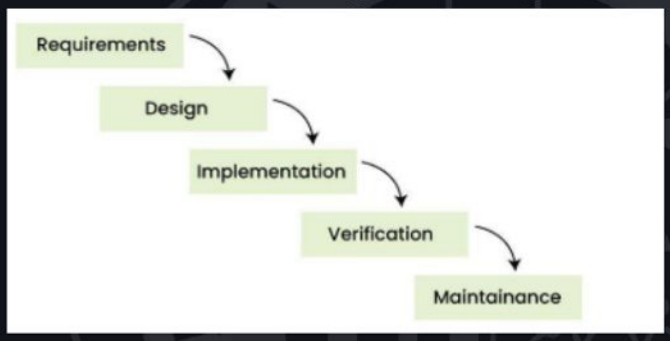
Aim: To understand DevOps: Principles, Practices, and DevOps Engineer Role and Responsibilities.

Description:

1. Waterfall Model

Description: A linear and sequential SDLC model where each phase is completed before the next begins.

Phases: Communication → Planning → Modeling → Construction → Deployment



Advantages:

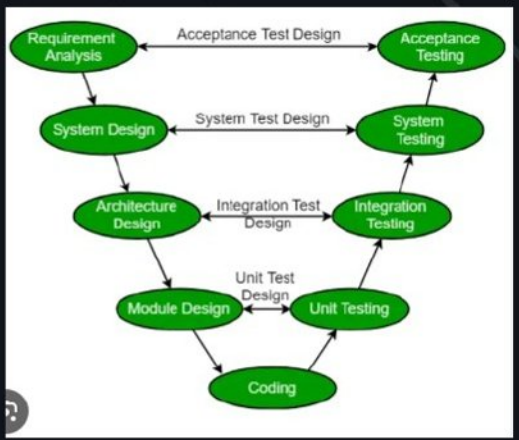
- Simple to understand and manage.
- Clearly defined phases and milestones.
- Good documentation.
- Suitable for stable requirements.

Disadvantages:

- Rigid and difficult to change.
- Requires complete requirements at the beginning.

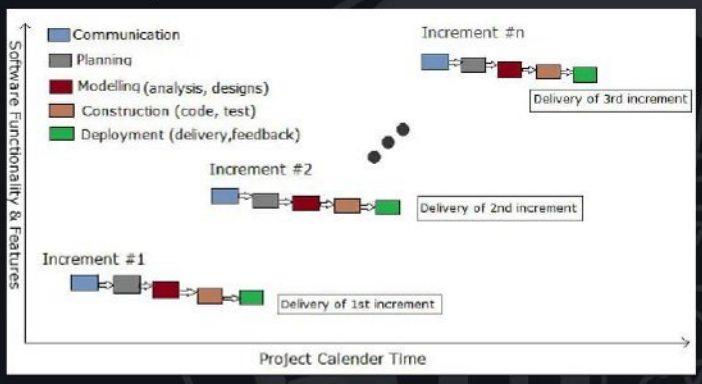
2. V-Model

Description: Extension of the Waterfall model where every development phase has a corresponding testing phase.



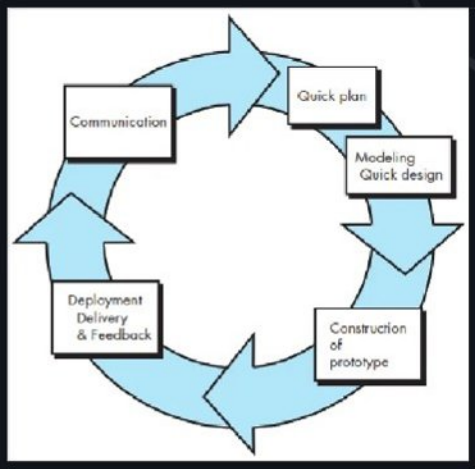
Advantages:
• Highly structured.
• Early test planning.
• Easier bug detection.
Disadvantages:
• Expensive.
• Less flexible.
• Changes are difficult.

3. Incremental Model
Description: Software is developed and delivered in small increments, each adding new functionality.



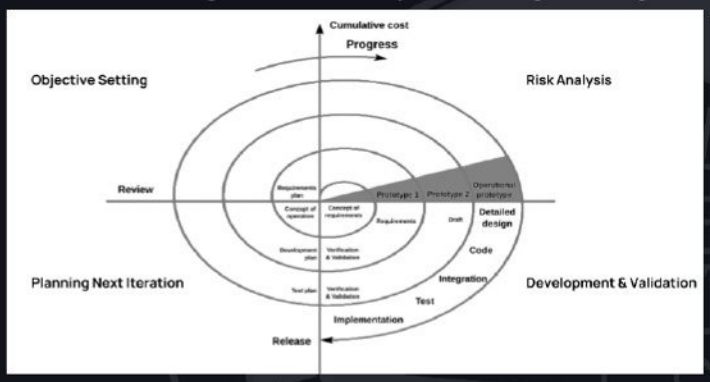
Advantages:
• Early delivery of working software.
• Easy to test each increment.
• Customer feedback can be incorporated.
Disadvantages:
• Requires good planning.
• Overall design must be well defined.

4. Prototype Model
Description: A prototype is built to understand customer requirements before developing the final product.



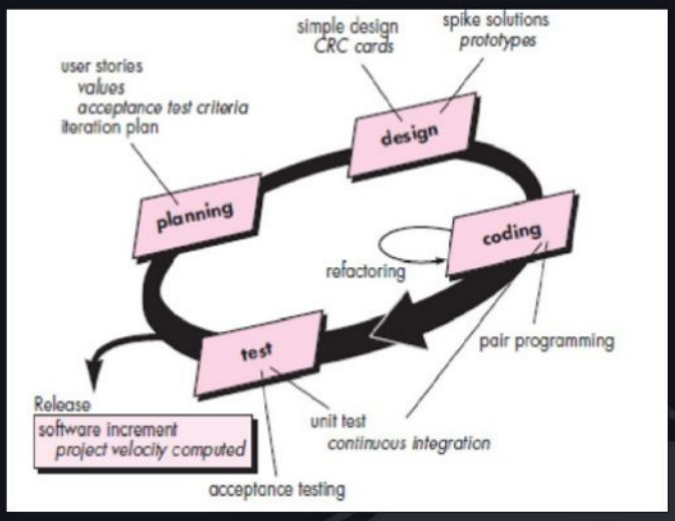
Advantages:
• Clarifies requirements.
• Early customer feedback.
• Reduces misunderstandings.
Disadvantages:
• Users may confuse the prototype with the final product.
• Can increase development cost and time.

5. Spiral Model
Description: A risk-driven model combining prototyping and iterative development.
Phases: Planning → Risk Analysis → Engineering → Customer Evaluation



Advantages:
• Excellent risk management.
• Continuous customer feedback.
• Suitable for large projects.
Disadvantages:
• Expensive.
• Complex to manage.

6. Agile Models
Extreme programming process
Description
The Agile model is an iterative and incremental software development model that delivers software in small releases. It focuses on customer collaboration, continuous feedback, and quick adaptation to changing requirements.



Advantages

- Responds quickly to changing requirements.
- Delivers working software frequently.
- Continuous customer feedback.
- High-quality software through regular testing.

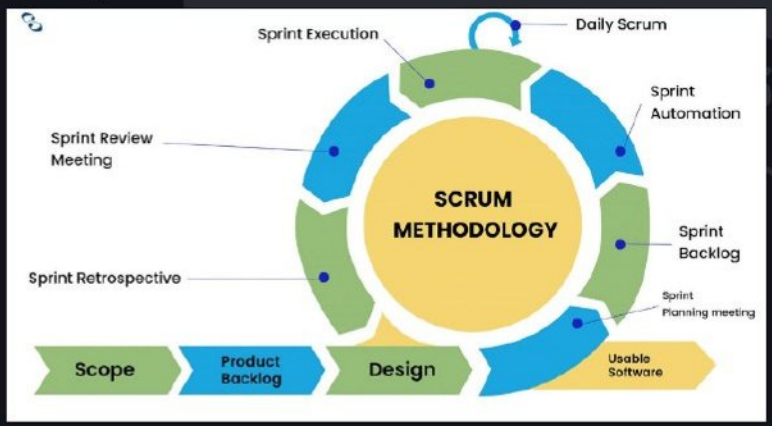
Disadvantages

- Difficult to scale for large teams.
- Requires experienced developers.
- Less documentation compared to Waterfall.

Scrum Model

Description

Scrum is an Agile framework in which software is developed in short iterations called Sprints (usually 2–4 weeks). It emphasizes teamwork, continuous feedback, and frequent delivery of working software.



Advantages

Delivers working software quickly.

- Easy to adapt to changing requirements.
- Continuous customer feedback.
- Improves team collaboration.

Disadvantages

- Requires experienced and disciplined teams.
- Scope changes can affect sprint planning.
- Frequent meetings may consume time.

DevOps

What is DevOps?

DevOps is a software development approach that combines **Development (Dev)** and **Operations (Ops)** to improve collaboration, automate workflows, and deliver software faster with higher quality and reliability.

Roles and Responsibilities of DevOps Engineer

- Automate software build, testing, and deployment processes.
- Manage CI/CD pipelines for continuous integration and delivery.
- Monitor application performance and system health.
- Manage cloud infrastructure and servers.
- Ensure security, backups, and disaster recovery.
- Collaborate with developers, testers, and operations teams.
- Troubleshoot production issues and improve system reliability.
- Use Infrastructure as Code (IaC) tools to manage infrastructure.

DevOps Lifecycle (7 C's)

1. Continuous Development

Developers continuously write and update code based on customer requirements using version control systems like Git.

2. Continuous Integration

New code is frequently integrated into a shared repository, where automated builds and tests detect errors early.

3. Continuous Testing

Automated testing ensures the application is functional, reliable, and free from defects before deployment.

4. Continuous Deployment

The tested application is automatically deployed to production or staging environments with minimal manual effort.

5. Continuous Monitoring

Applications and infrastructure are continuously monitored to detect performance issues, failures, and security threats.

6. Continuous Feedback

Feedback from users and monitoring tools is collected to improve future releases and enhance software quality.

7. Continuous Operations

Ensures applications remain available with minimal downtime through automation, maintenance, and infrastructure management.

Conclusion:

Different SDLC models are suitable for different types of software projects based on their requirements and development approach. Traditional models like Waterfall and V-Model work well for fixed requirements, while Agile and Scrum provide flexibility for changing needs. DevOps improves software development by promoting collaboration between development and operations teams, automating workflows, and enabling continuous integration, testing, deployment, and monitoring. This helps deliver high-quality software faster and more efficiently.

TM
CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

 Signature of the Faculty.....

DT LAB-2

Section: C1
Name:G.Susritha
Roll num: 160123733014

Aim: Explore the Version Control System tools for Source Code Management.

Description:

Version Control:
Version control, also known as source control, is the practice of tracking and managing changes to software code.

Version Control System (VCS) Tools:
Version Control System (VCS) tools are software applications that help developers manage, track, and coordinate changes to source code. Popular VCS tools include Git, GitHub, GitLab, Bitbucket, and SVN.

Source Code Management:
Source Code Management (SCM), also known as version control, is the practice of tracking and managing changes to software code. It utilizes centralized repositories to log modifications, enabling multiple developers to collaborate simultaneously, resolve conflicts, and safely revert to previous versions if errors occur.

Components of Version Control Systems:

Version Control Systems work using a few core concepts that help teams manage code changes and collaborate efficiently.

- Repository: A central location that stores all project files along with their complete change history and metadata like author and commit message.
- Revision: A specific saved version of a file or project, identified using a unique ID such as a hash or number.
- Branch: A separate copy of the codebase used to develop features or fix bugs without affecting the main code.
- Merging: The process of combining changes from one branch into another, which may sometimes require resolving conflicts.
- Commit: A snapshot of changes made to the codebase at a specific time, used to track and manage project history.

Types of version control systems:

1) Local VCS - Stores versions in a local database (eg, RCS)
2) Centralized VCS (CVCS) - A single server stores all versions, and clients check out files (eg, SVN, CVS)

3) Distributed VCS (DVCS) - Every developer has a complete copy of the repository (eg, Git, Mercurial)

Popular VCS Tools for Source Code Management (SCM)

1. **Git** – A distributed version control system widely used for tracking code changes and collaboration.
2. **GitHub** – A cloud-based platform for hosting Git repositories and collaborative software development.
3. **GitLab** – A Git-based platform that provides repository hosting along with built-in CI/CD features.
4. **Bitbucket** – A Git repository management platform developed by Atlassian with Jira integration.
5. **Apache Subversion (SVN)** – A centralized version control system used for managing source code versions.
6. **Mercurial** – A distributed version control system known for its simplicity and performance.

Conclusion:

Version Control Systems (VCS) are essential for managing source code efficiently by tracking changes, maintaining version history, and supporting collaboration among developers. Different types of VCS, such as Local, Centralized, and Distributed systems, meet different project needs, while tools like Git, GitHub, GitLab, Bitbucket, SVN, and Mercurial provide powerful features for source code management. Using VCS improves code quality, simplifies collaboration, and makes software development more organized and reliable.

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వినా భవ
1979

DT LAB-3

Section: C1
Name:G.Susritha
Roll num: 160123733014

Aim: Install git and create a GitHub account and to execute various GIT operations.

Description:

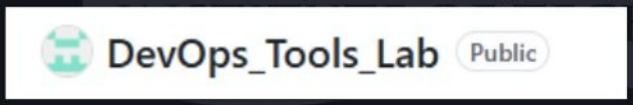
Git is a distributed version control system used to track changes in source code, while GitHub is a cloud-based platform for hosting Git repositories and collaborating on software projects. In this experiment, Git was installed, a GitHub account was created by signing in with a Google account, a Python program was managed using Git, and various Git operations such as initialization, commit, push, and clone were performed successfully.

Procedure:

1. Downloaded and installed Git for Windows from the official Git website.
2. Verified the installation by opening Command Prompt and checking the Git version.

```
C:\Users\student>git --version
git version 2.55.0.windows.2
```

3. Opened the GitHub website.
4. Created a GitHub account by selecting Continue with Google and signed in using a Google account.
5. Completed the GitHub account setup process by choosing a username and configuring the profile.
6. Created a new repository named DevOps_Tools_Lab on GitHub.



7. Developed a Python program (matrix_multiplication.py) for matrix multiplication.
8. Executed the program using:
   python matrix_multiplication.py

```
C:\014\week3>python matrix_multiplication.py
Result:
[19, 22]
[43, 50]
```

9. Initialized a local Git repository:
   git init

```
C:\014\week3>git init
Reinitialized existing Git repository in C:/014/week3/.git/
```

10. Added the project files to the staging area:
    git add .

```
C:\014\week3>git add .
```

11. Attempted to commit the changes:
    git commit -m "added matrix multiplication"

```
C:\014\week3>git commit -m "added matrix multiplication"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'student@cselab5-10.(none)')
```

12. Configured the Git username and email because Git requested the user identity:
    git config --global user.email "gudimetlasusritha@gmail.com"
    git config --global user.name "SusrithaG1908"

```
C:\014\week3>git config --global user.email "gudimetlasusritha@gmail.com"

C:\014\week3>git config --global user.name "SusrithaG1908"
```

13. Successfully committed the project:
    git commit -m "added matrix multiplication"

```
C:\014\week3>git commit -m "added matrix multiplication"
[master (root-commit) 994902d] added matrix multiplication
 1 file changed, 24 insertions(+)
 create mode 100644 matrix_multiplication.py
```

14. Renamed the default branch to main:
    git branch -M main

```
C:\014\week3>git branch -M main
```

15. Connected the local repository with the GitHub repository:
    git remote add origin https://github.com/SusrithaG1908/DevOps_Tools_Lab.git

```
C:\014\week3>git remote add origin https://github.com/SusrithaG1908/DevOps_Tools_Lab.git
```

16. Pushed the local repository to GitHub:
    git push origin main

Signature of the Faculty……...

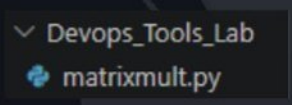
```
C:\014\week3>git push origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 428 bytes | 428.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/SusrithaG1908/DevOps_Tools_Lab.git
 * [new branch]      main -> main
```

17. Completed authentication through the browser using the GitHub account when prompted.
18. Cloned another GitHub repository:
    git clone https://github.com/pragnagosula/Devops_Tools_Lab.git

```
C:\014\week3>git clone https://github.com/pragnagosula/Devops_Tools_Lab.git
Cloning into 'Devops_Tools_Lab'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

19. Opened the cloned repository and verified its contents using:
    dir

```
C:\014\week3>cd Devops_Tools_Lab
```



Conclusion:

The experiment was completed successfully. Git was installed, a GitHub account was created and accessed using a Google account, and essential Git operations including repository initialization, staging, committing, pushing to GitHub, and cloning a repository were performed successfully. This experiment demonstrated the basic workflow of version control using Git and GitHub.

DT LAB-4

Section: C1
Name:G.Susritha
Roll num: 160123733014

Aim:. To install and configure Docker Desktop and Jenkins on a Windows system.

Description:

Docker

Docker is an open-source containerization platform that enables developers to package applications and their dependencies into lightweight, portable containers. These containers can run consistently across different environments, making application deployment faster and more reliable. Docker simplifies software development, testing, and deployment by eliminating environment-related issues.

Jenkins

Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Deployment (CD). It automates tasks such as building, testing, and deploying applications. Jenkins supports hundreds of plugins, allowing integration with version control systems, build tools, testing frameworks, and cloud platforms.

Procedure:

Docker Installation

1. Downloaded **Docker Desktop for Windows** from the official Docker website.



2. Executed the Docker Desktop installer.
3. Accepted the license agreement and enabled the **Use WSL 2 instead of Hyper-V** option (if prompted).
4. Completed the installation and restarted the system.
5. Launched Docker Desktop and waited for the Docker Engine to start successfully.
6. Verified the Docker installation using:
   docker --version

```
C:\Windows\System32>docker --version
Docker version 29.6.1, build 8900f1d
```

7. Displayed Docker system information using:
   docker info

```
C:\Windows\System32>docker info
Client:
 Version:    29.6.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.98.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.27.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.35.0-desktop.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
```

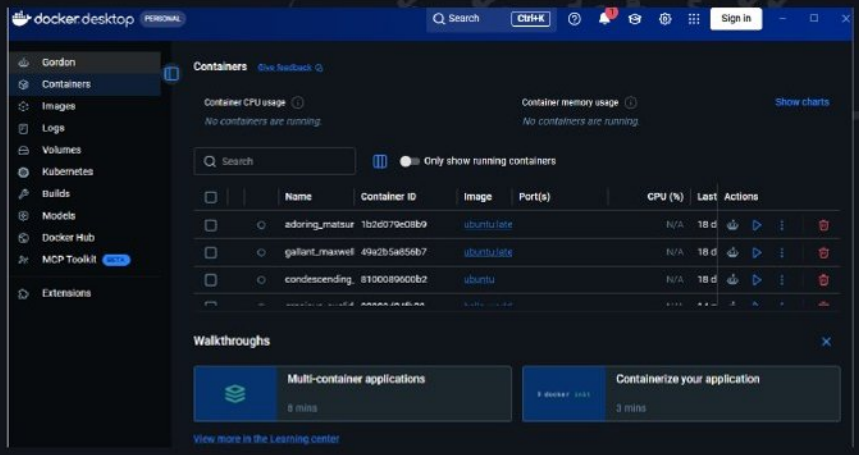
8. Tested the Docker installation by running:
   docker run hello-world

```
C:\Windows\System32>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:7f4da0fc94bcece205a8c0b6f4d11c8196924654ffe5c4d1aa439b7f632048b2
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

9. Verified the running Docker containers using:
   docker ps

```
C:\Windows\System32>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```



Jenkins Installation

1. Downloaded **Java Development Kit (JDK 21)** from the official Oracle website.

Windows x64 Installer	184.51 MB	https://download.oracle.com/java/25/archive/jdk-25.0.3_windows-x64_bin.exe (sha256)

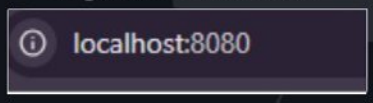
2. Installed the JDK by running the Windows installer.
3. Verified the Java installation using:
   java --version

```
C:\Windows\System32>java --version
java 21.0.11 2026-04-21 LTS
Java(TM) SE Runtime Environment (build 21.0.11+9-LTS-211)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.11+9-LTS-211, mixed mode, sharing)
```

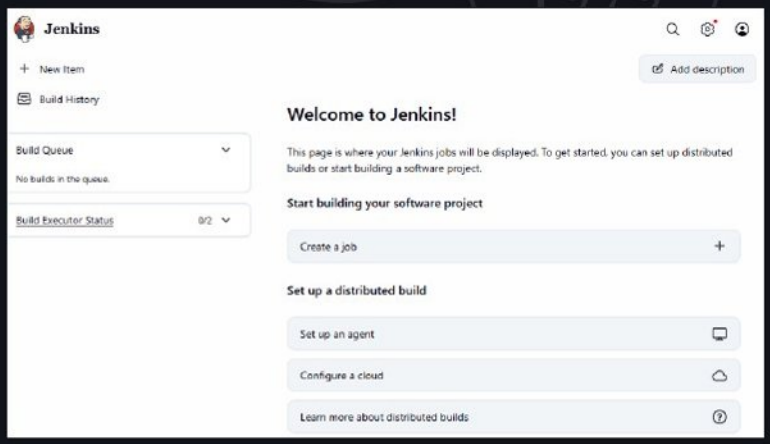
4. Downloaded the **Jenkins Windows Installer (.msi)** from the official Jenkins website.
5. Executed the Jenkins installer and completed the installation wizard.
6. Allowed Jenkins to be installed as a **Windows Service**.
7. Opened the **Services** application (services.msc) and verified that the Jenkins service was running.
8. Opened a web browser and accessed Jenkins using:
   http://localhost:8080
9. Retrieved the initial administrator password from:
   C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword.

```
C:\Windows\System32>type C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
aa3cceba688b4606a0a687fe5509b5c5
```

10. Entered the administrator password on the Jenkins Unlock page.
11. Selected **Install Suggested Plugins** to install the recommended Jenkins plugins.
12. Created the first administrator account by providing the required username, password, and email address.
13. Completed the Jenkins setup and accessed the Jenkins Dashboard through:
    http://localhost:8080



14. Verified that the Jenkins Dashboard loaded successfully, confirming that Jenkins was installed and configured correctly.



 Signature of the Faculty……...

Conclusion:

The installation of Docker and Jenkins was completed successfully. Docker provides a reliable platform for creating and managing containers, while Jenkins enables automated software build, testing, and deployment processes. Together, Docker and Jenkins form a fundamental DevOps toolchain that improves development efficiency, ensures consistent deployment environments, and supports Continuous Integration and Continuous Deployment (CI/CD) practices.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY TM
స్వయం తేజస్వీన్ భవ
1979

Signature of the Faculty.....

DT LAB-5

Aim: To understand Jenkins Master-Slave Architecture and scale your Jenkins standalone implementation by implementing slave nodes.

Description:

Jenkins

Jenkins is an open-source automation tool written in Java with plugins built for Continuous Integration purposes. Jenkins is used to build and test your software projects continuously making it easier for developers to integrate changes to the project, and making it easier for users to obtain a fresh build. It also allows you to continuously deliver your software by integrating with a large number of testing and deployment technologies.

With Jenkins, organizations can accelerate the software development process through automation. Jenkins integrates development life-cycle processes of all kinds, including build, document, test, package, stage, deploy, static analysis and much more.

Jenkins achieves Continuous Integration with the help of plugins. Plugins allow the integration of Various DevOps stages. If you want to integrate a particular tool, you need to install the plugins for that tool. For example, Git, Maven 2 project, Amazon EC2, HTML publisher, etc.

Jenkins Master-Slave Architecture

The Jenkins Master-Slave architecture (now commonly referred to as the Controller-Agent architecture) is a distributed model designed to offload job execution from the main Jenkins server. When a standalone Jenkins instance grows, running heavy CI/CD pipelines concurrently can exhaust the CPU and memory of the master node, causing performance drops or complete system crashes. Scaling a standalone Jenkins instance using physical or virtual slave nodes ensures high availability, parallel task execution, and cross-platform ecosystem builds.

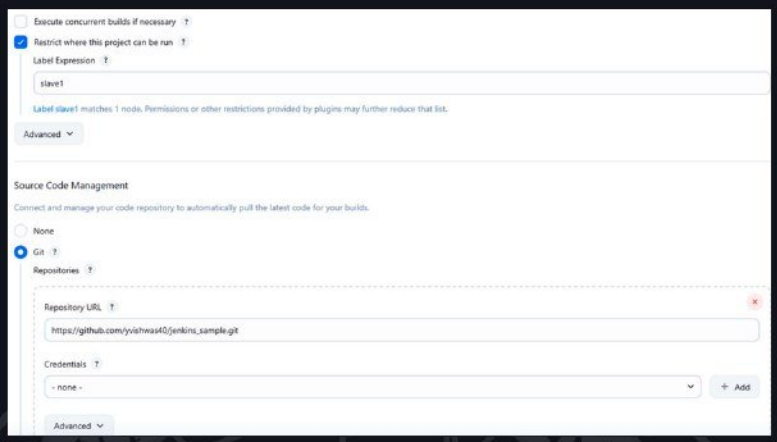
Procedure:

1. Install Java and Jenkins on the master system.
2. Start Jenkins and configure initial setup.
3. Create a new slave node in Jenkins and assign label slave1.



4. Download the agent.jar file and connect the slave node using the provided secret key and command.

5. Verify the slave node is in connected state.
6. Create a freestyle job named Slave_function.
7. Configure the job to pull code from GitHub repository https://github.com/Sahithi-2408/jenkins_sample.git.

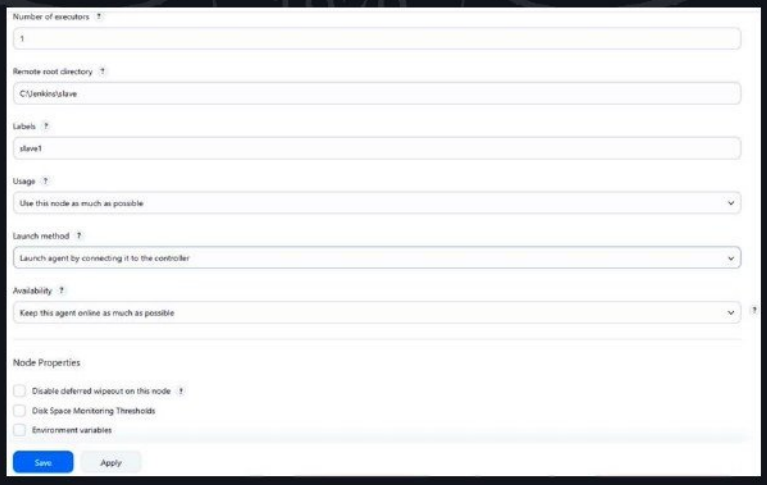


8. Add a build step to run the Python program app.py.
9. Restrict the job to run only on slave node using label slave1.
10. Save and build the job.

```
8ad9405 -name Slave1 -webSocket -workDir "C:\Jenkins\slave"
Sept 11, 2025 10:59:44 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using C:\Jenkins\slave\remoting as a remoting work directory
Sept 11, 2025 10:59:44 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to C:\Jenkins\slave\remoting
Sept 11, 2025 10:59:44 PM hudson.remoting.Launcher createEngine
INFO: Setting up agent: Slave1
Sept 11, 2025 10:59:44 PM hudson.remoting.Engine startEngine
INFO: Using Remoting version: 3309.v27b_9314fd1a_4
Sept 11, 2025 10:59:44 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using C:\Jenkins\slave\remoting as a remoting work directory
Sept 11, 2025 10:59:45 PM hudson.remoting.Launcher$CuiListener status
INFO: WebSocket connection open
Sept 11, 2025 10:59:45 PM hudson.remoting.Launcher$CuiListener status
INFO: Connected
```

Execution Steps:

- Open Jenkins dashboard.
- Navigate to Manage Jenkins → Nodes → New Node.
- Create node Slave1 with label slave1.



- Download agent.jar and connect using the given command.
- Confirm the slave is online (as shown in monitoring data).
- Create a freestyle project or pipeline.
- Configure the job with Restrict where this project can be run → slave1.
- Add a build step to print output: echo Hello from Slave1
- Build the project and verify execution on Slave1.

Sample Output:

```
Running as SYSTEM
Building remotely on Slave1 (slave1) in workspace C:\Jenkins\slave\workspace\Slave_function
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/yvishwas40/jenkins_sample.git
 > git.exe init C:\Jenkins\slave\workspace\Slave_function # timeout=10
Fetching upstream changes from https://github.com/yvishwas40/jenkins_sample.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.45.1.windows.1'
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision ab55c0108c1b232a25bb17df6ff4264424e33ef0 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f ab55c0108c1b232a25bb17df6ff4264424e33ef0 # timeout=10
Commit message: "Update app.py"
 > git.exe rev-list --no-walk ab55c0108c1b232a25bb17df6ff4264424e33ef0 # timeout=10
[Slave_function] $ cmd /c call C:\Users\yvish\AppData\Local\Temp\jenkins5969414529213295698.bat

C:\Jenkins\slave\workspace\Slave_function>python app.py
This is sample Jenkins folder

C:\Jenkins\slave\workspace\Slave_function>exit 0
Finished: SUCCESS
```

Conclusion:

By configuring Jenkins master and slave architecture, distributed builds can be achieved. This setup enables load distribution, better scalability, and efficient utilization of resources. Slave nodes can be targeted with labels, ensuring specific jobs run on specific environments.

Docker Architecture and Lifecycle

AIM: *To understand Docker architecture and the container lifecycle, install Docker, and execute Docker commands to manage images and interact with containers.*

Objectives:

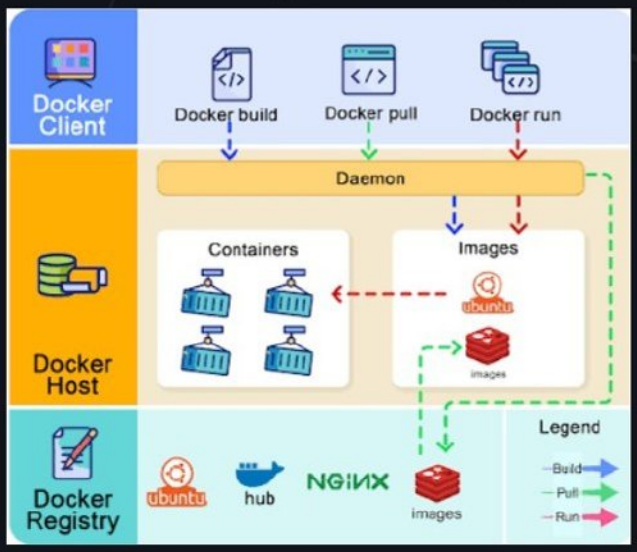
- *To understand the fundamentals of containerization using Docker.*
- *To study the architecture and components of Docker.*
- *To understand the lifecycle of a Docker container.*
- *To install and configure Docker on a system.*
- *To execute commonly used Docker commands for image and container management.*
- *To interact with running containers and observe their behavior.*

Description:

Docker is an open-source containerization platform that enables applications and their dependencies to be packaged into lightweight and portable containers. Unlike traditional virtual machines, containers share the host operating system kernel, making them more efficient in terms of resource utilization and startup time. Docker helps developers:

- *Build applications consistently across environments.*
- *Simplify deployment processes.*
- *Improve scalability and portability.*
- *Isolate applications from one another.*
- *Reduce dependency-related issues.*

***Docker Architecture**:*



Signature of the Faculty.....

Docker follows a client-server architecture consisting of several components.

***Docker Client** :The Docker Client is the interface through which users execute Docker commands.*

- *docker run*
- *docker pull*
- *docker ps*

The client communicates with the Docker daemon to perform requested operations.

***Docker Daemon:** The Docker Daemon (dockerd) is responsible for:*

- *Building images.*
- *Managing containers.*
- *Handling Docker networks.*
- *Managing Docker volumes.*
- *Communicating with Docker clients.*

***Docker Host:** The Docker Host is the machine on which Docker is installed and running. It contains:*

• Docker Engine • Images • Containers Networks • Volumes

***Docker Registry** : A Docker Registry stores Docker images. Examples include:*

• Docker Hub • Private Docker Registry • Enterprise Container Registry

***Docker Images**:*

A Docker image is a read-only template used to create containers. Characteristics:

- *Contains application code and dependencies.*
- *Can be reused multiple times.*
- *Acts as a blueprint for containers.*
- *Stored locally or in registries.*

Examples: ubuntu, nginx, mysql, python

***Docker Containers**:*

A container is a running instance of a Docker image. Containers provide:

- *Process isolation.*

Signature of the Faculty.....

- *Application portability*
- *Lightweight execution.*
- *Rapid startup.*

Docker Container Lifecycle:

A Docker container passes through several states during its lifecycle.

Created: *The container has been created but is not running.*

Running: *The container is actively executing processes.*

Paused: *The container processes are temporarily suspended.*

Stopped: *The container is no longer running but still exists on the system.*

Removed: *The container has been deleted from the Docker host.*

Procedure :

1. *Download and install Docker Desktop or Docker Engine.*
2. *Verify that Docker has been installed successfully.*
3. *Study the major components of Docker architecture.*
4. *Understand the Docker container lifecycle.*
5. *Pull a Docker image from Docker Hub.*
6. *Create and start a container from the image.*
7. *View running and stopped containers.*
8. *Interact with a container using Docker commands.*
9. *Stop, restart, and remove containers.*
10. *Remove unused Docker images.*
11. *Record observations regarding image and container management.*

Execution:

Step 1: Install Docker

Download and install Docker Desktop from the official Docker website. After installation, start Docker Desktop and ensure the Docker service is running.

Signature of the Faculty.....

Step 2: Verify Docker Installation

Open Command Prompt, PowerShell, or Terminal and execute: docker --version A successful installation displays the installed Docker version.

Step 3: Verify Docker Service

Check whether Docker is functioning properly: docker info This command displays Docker Engine information and confirms communication with the Docker daemon.

Step 4: Download an Image

Pull the Ubuntu image from Docker Hub: docker pull ubuntu Docker downloads the required image layers and stores them locally.

Step 5: View Available Images

Display all downloaded images: docker images The command shows: • Repository name • Tag • Image ID • Creation date • Image size

Step 6: Create and Run a Container

Create and start a container using the Ubuntu image: docker run -it ubuntu bash Explanation: • -i : Interactive mode. • -t : Allocates a terminal. • ubuntu : Image name. • bash : Starts the Bash shell. The terminal prompt changes to indicate execution inside the container.

Step 7: Interact with the Container

Execute commands within the running container:

- *pwd*
- *ls*
- *whoami*

These commands demonstrate interaction with the containerized environment

Step 8: Exit the Container

Exit the running container: exit The container stops after exiting the shell session.

Step 9: View Container Lifecycle Status

Display all containers: docker ps -a

Observe the container status: • Up (Running) • Exited (Stopped)

This demonstrates lifecycle state transitions.

Step 10: Restart the Container

Start the stopped container again: docker start <container_id>

Signature of the Faculty.....

Verify the status: docker ps

Step 11: Access the Running Container

Connect to the container: docker exec -it bash This opens an interactive shell session within the container.

Step 12: Remove the Container

Stop the container if necessary: docker stop Remove it: docker rm This deletes the container from the Docker host.

Step 13: Remove the Image

Delete the image: docker rmi ubuntu This removes the image from local storage

Conclusion:

The experiment provided practical understanding of Docker architecture and containerization concepts. Docker images and containers were managed using standard Docker commands, and the complete container lifecycle—from creation to removal—was studied. The experiment demonstrated how Docker enables lightweight, portable, and efficient application deployment through container technology.

Signature of the Faculty.....

Docker file and Image Building

AIM: *To study Dockerfile instructions and use a Dockerfile to create a container image for a simple web application..*

Objectives:

- *To understand the purpose and structure of a Dockerfile.*
- *To study commonly used Dockerfile instructions.*
- *To create a simple web application that can be served using Nginx.*
- *To write a Dockerfile for the web application.*
- *To build a Docker image using the Dockerfile.*
- *To run a container from the newly created image.*
- *To access and verify the web application through a web browser*

Description:

A Dockerfile is a text file containing a sequence of instructions used by Docker to construct an image. Instead of manually configuring a container every time an application needs to be deployed, the required environment and application files can be described in a Dockerfile. A Dockerfile therefore acts as a reproducible set of instructions for creating a Docker image.

Dockerfile Instructions:

Instruction	Purpose
FROM	Specifies the base image for the new image
WORKDIR	Sets the working directory inside the image
COPY	Copies files from the build context into the image
ADD	Adds files or directories to the image
RUN	Executes a command while building the image
CMD	Specifies the default command executed when a container starts
ENTRYPOINT	Defines the primary executable for the container
EXPOSE	Documents the port intended to be used by the application
ENV	Defines environment variables
LABEL	Adds metadata to the image

Signature of the Faculty.....

Base Image:

The FROM instruction specifies the starting point for the image. For this experiment:

FROM nginx

The official Nginx image provides a ready-to-use web server environment.

***Copying Application Files**:*

The COPY instruction transfers files from the Docker build context into the image. For the sample application:

COPY index.html /usr/share/nginx/html

The HTML file is copied into the default directory from which Nginx serves static web content.

***Docker Image Building Process**:*

The Docker image is created using:

docker build -t myapp .

Here:

- *docker build starts the image-building process*
- *-t myapp assigns the name myapp to the image.*
- *. specifies the current directory as the build context.*

During the build, Docker reads the Dockerfile and executes its instructions to create the image.

***Running the Image as a Container**:*

After building the image, a container can be created using:

docker run -p 8080:80 myapp

The port mapping means: Host Port 8080 → Container Port 80

Therefore, requests sent to port 8080 on the host are forwarded to Nginx running on port 80 inside the container

Procedure :

1. Verify that Docker is installed and running.

2. Create a directory for the sample web application.

3. Create an HTML file containing the web-page content.

4. Create a file named Dockerfile in the application directory.

5. Add appropriate Dockerfile instructions.

6. Build a Docker image from the Dockerfile.

7. Verify that the newly created image is available locally.

Signature of the Faculty.....

8. Create and run a container using the image.

9. Map the container's web-server port to a port on the host system.

10. Open the application in a web browser and verify its output.

Execution:

Step 1: Create the Application Directory

Open PowerShell or Terminal and create a directory for the web application:

mkdir myapp

cd myapp

```
    Directory: C:\Users\Amol Pathak\Documents


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        07-10-2026     16:54                myapp
```

Step 2: Create the HTML File

Create a file named index.html.

For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>My Docker App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>This web page is running inside a Docker container.</p>
</body>
</html>
```

Step 3: Create the Dockerfile

Inside the same directory, create a file named: Dockerfile

The Dockerfile should contain:

FROM nginx

COPY index.html /usr/share/nginx/html

The first instruction selects Nginx as the base image, while the second copies the application's HTML file into the Nginx web directory. The reference experiment uses this same minimal Dockerfile structure for the sample application.

Signature of the Faculty.....

```
    Directory: C:\Users\Amol Pathak\Documents\myapp


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         07-10-2026     16:59             49 Dockerfile
-a----         07-10-2026     16:57            197 index.html
```

Step 4: Verify Docker

Before building the image, verify that Docker is available: docker --version

Optionally, obtain detailed Docker information using: docker info

Step 5: Build the Docker Image

From the directory containing the Dockerfile, execute:

docker build -t myapp .

Docker reads the Dockerfile and generates the myapp image. The build process should indicate successful completion. The reference execution shows the image being successfully built and tagged as myapp:latest. Figure 3: Successfully building the Docker image

```
[+] Building 20.1s (7/7) FINISHED                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.1s
 => => transferring dockerfile: 86B                                                 0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                     3.3s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [internal] load build context                                                   0.1s
 => => transferring context: 236B                                                   0.0s
```

Step 6: Verify the Image

List the locally available Docker images:

docker images

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
myapp:latest	d714a8d1e386	240MB	63.9MB	
postgis/postgis:15-3.3	a2fc46b52819	825MB	206MB	U

Step 7: Run the Container

Start a container from the newly created image:

docker run -p 8080:80 myapp

Here, port 8080 of the host is mapped to port 80 of the Nginx container. The reference experiment uses the same port mapping to run the myapp image.

Signature of the Faculty.....

```
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/07 11:39:12 [notice] 1#1: using the "epoll" event method
2026/10/07 11:39:12 [notice] 1#1: nginx/1.31.6
2026/10/07 11:39:12 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/10/07 11:39:12 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/10/07 11:39:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/10/07 11:39:12 [notice] 1#1: start worker processes
2026/10/07 11:39:12 [notice] 1#1: start worker process 29
2026/10/07 11:39:12 [notice] 1#1: start worker process 30
2026/10/07 11:39:12 [notice] 1#1: start worker process 31
2026/10/07 11:39:12 [notice] 1#1: start worker process 32
2026/10/07 11:39:12 [notice] 1#1: start worker process 33
2026/10/07 11:39:12 [notice] 1#1: start worker process 34
```

Step 8: Access the Web Application

Open a web browser and enter: http://localhost:8080

The HTML page created in Step 2 should be displayed. This confirms that:

1. The Dockerfile was processed successfully.

2. The Docker image was created.

3. A container was started from the image.

4. Nginx is serving the HTML file.

5. The application is accessible through the mapped host port.

Step 9: Verify the Running Container

Open another terminal and execute: docker ps

This displays the currently running containers. The output should show the myapp container and its port mapping

```
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                      NAMES
503f98f95df4   myapp     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    agitated_proskuria
ova
```

Conclusion:

The experiment provided practical understanding of Dockerfile-based image creation. The FROM and COPY instructions were used to construct an Nginx-based image containing a sample HTML application. The resulting image was built and executed as a container, demonstrating how Dockerfiles provide a repeatable method for packaging and deploying web applications.

Signature of the Faculty.....

Kubernetes Cluster

AIM: *Deploy a containerized application on Kubernetes cluster*

Description:

Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications. Docker is used to package an application and its dependencies into a container image. In this experiment, a simple HTML web application is created and packaged into a Docker image using Nginx as the web server. The image is loaded into a local Kubernetes cluster managed by Minikube. A Kubernetes Deployment is created with two replicas to run the application in two Pods. Finally, a NodePort Service is configured to expose the application and make it accessible through a web browser

Execution:

Step 1: Start the Kubernetes cluster

Start Minikube and verify that the cluster is running. minikube start --driver=docker kubectl get nodes

Step 2: Create the application

Create a project directory and an index.html file containing a simple webpage with the message "Hello from Kubernetes!".

Step 3: Create the Dockerfile

Create a file named Dockerfile with the following instructions:

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

Step 4: Build the Docker image

Build the application image using the following command:

docker build -t k8s-demo:v1 .

Verify that the image was created successfully:

docker images k8s-demo

Signature of the Faculty.....

Step 5: Load the image into Minikube

Make the Docker image available to the Kubernetes cluster.

minikube image load k8s-demo:v1

Step 6: Create a Kubernetes Deployment

Create a Deployment with two replicas of the application.

kubectl create deployment k8s-demo --image=k8s-demo:v1 --replicas=2

Verify the Deployment: kubectl get deployments

Step 7: Verify the Pods

Check whether both application Pods are running.

kubectl get pods

Both Pods should show Running status and 1/1 readiness.

Step 8: Expose the application

Create a NodePort Service to expose the application on port 80.

kubectl expose deployment k8s-demo --type=NodePort --port=80

Verify the Service:

kubectl get services

Step 9: Access the application

Run the following command to access the application through a browser:

minikube service k8s-demo

The webpage displays the message "Hello from Kubernetes!", confirming that the application is accessible.

Conclusion:

Thus, the containerized web application was successfully deployed on a Kubernetes cluster using Docker and Minikube. A Deployment was created with two replicas, the Pods were verified to be running, and a NodePort Service was used to expose the application. The application was accessed successfully through a web browser, demonstrating the basic process of container deployment and orchestration using Kubernetes.

Signature of the Faculty.....

Puppet

AIM: *To install and configure Pull based Software Configuration Management and provisioning tools using Puppet.*

Description:

Puppet is an open-source configuration management and provisioning tool used to automate the installation, configuration, and maintenance of systems. It follows a declarative approach in which the desired state of system resources is defined using Puppet manifests. In the pull-based model, the Puppet Agent periodically contacts the Puppet Server to retrieve its configuration catalog. The agent applies the catalog locally to ensure that the system matches the desired state. Puppet also supports idempotency, meaning repeated application of the same configuration does not make unnecessary changes when the system is already in the desired state. In this experiment, Puppet Agent and Puppet Server are installed in Ubuntu running on WSL. A simple Puppet manifest is created to ensure that a directory exists. The manifest is applied and tested for idempotency. Finally, the Puppet Agent contacts the Puppet Server, retrieves a catalog, and applies the configuration.

Execution:

Step 1: Update the package repositories

Update the Ubuntu package lists before installing Puppet.

```
sudo apt update
```

Step 2: Install Puppet Agent

Install the Puppet Agent package.

```
sudo apt install puppet-agent
```

Verify the installation:

```
puppet --version
```

Step 3: Install Puppet Server

Install the Puppet Server package.

```
sudo apt install puppetserver
```


Signature of the Faculty.....

Step 4: Start and verify the Puppet Server

Start the Puppet Server service.

sudo systemctl start puppetserver

Check its status:

sudo systemctl status puppetserver

The service should display active (running).

Step 5: Configure the Puppet Server hostname

Check the server hostname configured for the agent.

sudo puppet config print server --section agent

The configured hostname is puppet. Add the hostname mapping to /etc/hosts so the agent can resolve it locally.

echo "127.0.0.1 puppet" | sudo tee -a /etc/hosts

Verify the hostname resolution:

getent hosts puppet

Step 6: Verify the Puppet Server port

Check whether the Puppet Server is listening on port 8140

sudo ss -ltnp | grep 8140

The output should show a listening service on port 8140.

Step 7: Create a Puppet manifest

Create a manifest named first.pp.

nano first.pp

Enter the following Puppet code:

file { '/tmp/puppet-demo': ensure => directory, }

Save the file and exit the editor.

Step 8: Apply the Puppet manifest

Execute the manifest using the Puppet command.

sudo puppet apply first.pp

Signature of the Faculty.....

Puppet creates the directory /tmp/puppet-demo if it does not already exist.

Step 9: Verify the resource

Check whether the directory has been created.

```
ls -ld /tmp/puppet-demo
```

The output should display the directory and its permissions.

Step 10: Verify idempotency

Apply the same manifest again.

```
sudo puppet apply first.pp
```

Since the directory already exists in the desired state, Puppet should report that no resource changes are required.

Step 11: Execute the Puppet Agent

Run the Puppet Agent in test mode to initiate communication with the Puppet Server.

```
sudo puppet agent --test
```

The agent requests a catalog from puppet:8140, retrieves the catalog, and applies the configuration successfully.

Conclusion:

Thus, Puppet was successfully installed and configured as a pull-based Software Configuration Management and provisioning tool. A Puppet manifest was created and applied successfully, idempotency was verified, and communication between the Puppet Agent and Puppet Server was demonstrated.

Signature of the Faculty.....

Software Configuration Management

AIM: *To learn Software Configuration Management and provisioning using Puppet Blocks(Manifest, Modules, Classes, Function)*

Description:

Puppet provides a declarative language for defining the desired state of system resources. Its main building blocks help organize and reuse configuration code.

- ***Manifest****: A file containing Puppet code, usually with the .pp extension, that describes the desired configuration of resources.*
- ***Module:*** *A reusable collection of manifests and other supporting files organized in a standard directory structure.*
- ***Class:*** *A named block of Puppet code that groups related resources and can be included wherever required.*
- ***Function:*** *A reusable operation that accepts inputs and returns a value or performs a supported operation. Puppet provides built-in functions such as upcase() for transforming strings.*

In this experiment, a Puppet module is created with a class that manages a file. A manifest includes the class, and the built-in upcase() function is used to generate the file content. The configuration is then applied and verified.

Execution:

Step 1: Create the project directory

Create a directory for the Puppet experiment and move into it.

```
mkdir -p ~/puppet-blocks-lab/modules/webdemo/manifests
cd ~/puppet-blocks-lab
```

Step 2: Create a Puppet module

A module organizes related configuration code. The webdemo module contains a manifests directory and an init.pp file.

The directory structure is:

Signature of the Faculty.....

```
puppet-blocks-lab/
├── site.pp
└── modules/
    └── webdemo/
        └── manifests/
            └── init.pp
```

Step 3: Create a class inside the module

Create the module's main manifest.

```
nano modules/webdemo/manifests/init.pp
```

Add the following code:

```
class webdemo {
    file { '/tmp/puppet-blocks-demo.txt':
        ensure => file,
        content => upcase('puppet blocks demonstration'), } }
```

Save the file and exit the editor. Here, class webdemo defines a class, while the file resource ensures that the specified file exists with the required content.

Step 4: Understand the function

The built-in Puppet function upcase() converts a string into uppercase letters. For example:

```
upcase('puppet blocks demonstration')
```

The result is:

```
PUPPET BLOCKS DEMONSTRATION
```

This result is assigned to the file's content attribute.

Step 5: Create the main manifest

Create the main manifest that includes the module's class.

```
nano site.pp
```

Add the following code:

```
include webdemo
```

Save the file and exit the editor. The include statement declares the class so Puppet can apply the resources defined inside it.

Signature of the Faculty.....

Step 6: Apply the Puppet configuration

Run Puppet with the module path specified.

sudo puppet apply --modulepath=modules site.pp

Puppet locates the webdemo module, evaluates its class, executes the function, and creates the configured file.

Step 7: Verify the output

Display the generated file content.

cat /tmp/puppet-blocks-demo.txt

Expected output:

PUPPET BLOCKS DEMONSTRATION

Step 8: Verify idempotency

Apply the same configuration again.

sudo puppet apply --modulepath=modules site.pp

Puppet should report that no resource changes are required because the file already matches the desired state

Conclusion:

Thus, Software Configuration Management and provisioning using Puppet blocks were studied and implemented successfully. A manifest was used to include a class, a module was created to organize reusable configuration code, and the built-in upcase() function was used to generate the required file content. The experiment demonstrated the modular, reusable, and idempotent nature of Puppet configuration management.

Signature of the Faculty.....